

Group symbol: **3**

Team: **1**

Project title: **Resource Allocation App**

Team members (*filled by PM, Team Leader*):

No	Name	Surname	Student ID	Role
1	Carmelo Aidan	Romero	293936	<i>PM, Team Leader</i>
2	Álvaro	Puebla	293867	<i>Team member</i>
3	David	Sosa	293885	<i>Team member</i>
4	Diego	Ceballos	293943	<i>Team member</i>
5	Diego	García	293939	<i>Team member</i>
6	Óscar	López	293871	<i>Team member</i>

1. Elaboration of application concept (S1)

1.1. Project (business) goals

Present as a tree of the project's objectives, i.e., decomposition of general project objective into specific ones. A goal should be:

- relevant to the project,
- important from the perspective of the end-user,
- achievable,
- measurable,
- realistic (achievable in terms of constraints: cost and time-oriented, technological, organizational, human resources, legal).

General objective: Build a university-wide system that lets users reserve, borrow, rent or book any campus resource (books, rooms, equipment, vehicles, study kits), manage approvals and loans, enable physical check-in/out (QR/ISBN/barcode), and show real-time availability and status across campus.

The **main goals** are:

1. Unified resource access

1.1 Provide a **single interface** for discovering and reserving/lending books, rooms, and equipment.

- Measurable: 90% of pilot resource types available at MVP launch.
- Time-bound: Pilot in 6–9 weeks.

1.2 **Support search & filters** (by type, department, availability).

- Measurable: Average search response < 500 ms under test load.

2. Reliable loan & reservation lifecycle

2.1 Implement reservation, approval, check-out, and return flows for different resource types.

- Measurable: 98% success for end-to-end flows in QA tests.

2.2 Enforce rules (max loan time, quotas, training requirements).

- Measurable: Rules correctly enforced in 100% of randomized tests.

3. Accurate physical confirmation

3.1 Enable QR/barcode/ISBN scanning for check-in/out and returns.

- Measurable: $\geq 95\%$ successful scan confirmation rate on supported devices.

3.2 Auto-handle overdue and no-show workflows (notifications, holds).

- Measurable: Overdue detection within 1 minute of scheduled end; notifications dispatched reliably.

4. Visibility & operational control

4.1 Provide a campus map/floorplan and availability indicators (green: available, red: occupied/on-loan, yellow: maintenance/cleaning).

- Measurable: Map status reflects system state within 30s for 90% of events.

4.2 Deliver manager dashboards and usage/exportable reports for billing/audit.

- Measurable: Generate per-resource usage CSV for any date range.

5. Deployable, secure MVP

5.1 Docker-based MVP deployable on one small cloud VM.

- Measurable: Successful documented deployment performed by PM.

5.2 Meet baseline security & privacy (HTTPS, auth, retention policy).

- Measurable: Security checklist completed; no high-severity findings at launch

1.2. Identification of project's internal and external Stakeholders

Identify the role of each **internal** stakeholder in the project, along with a textual description.

Symbol	Name	Role	Description
I1	Carmelo A. Romero	Cross-functional team member	Leads the team, coordinates stakeholders, owns schedule and scope, primary contact for sponsor/IT; also contributes across project tasks as needed.
I2	Álvaro Puebla	Cross-functional team member	Contributes across planning, development, testing, deployment and documentation as required.
I3	David Sosa	Cross-functional team member	Contributes across planning, development, testing, deployment and documentation as required.
I4	Diego Ceballos	Cross-functional team member	Contributes across planning, development, testing, deployment and documentation as required.
I5	Diego García	Cross-functional team member	Contributes across planning, development, testing, deployment and documentation as required.
I6	Óscar López	Cross-functional team member	Contributes across planning, development, testing, deployment and documentation as required.

Identify the role of each **external** stakeholder in the project, along with a textual description.

Symbol	Name	Role	Description
E1	Students	Primary end users	Search, reserve, borrow materials and rooms; need simple mobile flows and notifications.
E2	Faculty & Researchers	Power users	Reserve recurring rooms, request equipment for classes/research, need reports and billing integration.
E3	Library Staff	Resource managers	Manage book inventories, loans, returns, overdue policies; process approvals and holds.
E4	Facility Managers	Room owners	Approve room bookings, mark rooms unavailable (events, maintenance), provide floorplans.
E5	Lab/Equipment Managers	Equipment owners	Enforce training-based access, schedule maintenance, and mark equipment status.
E6	University IT	Integration & security	SSO, directory integration, network policy, hosting support.
E7	Students' Union / Admin	Billing & policy	Interested in usage data, fines, and reporting for charges or resource scheduling.

1.3. Domain description

- Description of phenomena in the application domain that can be observed.
- Definitions of concepts that abstract these phenomena.
- The concepts include:
 - entities – things that one can “distinguish” from the fraction of reality,
 - relationships – either properties (unary relationships of entities) that yield some information about them or relationships between two or more entities (binary or n -ary relations),
 - events – things that can happen (occur).
- Use UML class diagram to depict entities (class with attributes), relationships (associations).
- This diagram **must** be supplemented with the list of entities, relationships and events along with their description in natural language.

Domain phenomena:

- **Multi-type resource lifecycle:** Reservation/loan **flows differ per type** (rooms: timed reservation; books: loan checkout/return; equipment: reservation + check-in/out, possibly training prerequisites).
- **Hold/Queue mechanics:** When an item is on loan, users can place holds; next eligible user is notified.
- **Physical confirmation:** Barcode/ISBN/QR scans confirm checks and returns.
- **Status propagation:** Check-in/out, **maintenance**, and cleaning events **update availability** and map visualizations near real-time.
- **Overdue & fines:** Items unreturned trigger overdue events, notifications, and possibly fines or account blocks.

Definition of concepts:

1. User — campus account (student, faculty, staff). (user_id, name, email, role, department, account_status)
2. Department — organizational owner of resources. (dept_id, name, contact)
3. Resource (abstract) — any lendable or bookable item. (resource_id, type, name, location_id, status, department_id)
4. Subtypes: Book (ISBN, shelf_location, copies_available), Room (capacity, floorplan_position), Equipment (serial_number, requires_training), Vehicle, etc.
5. Reservation — a booking request for a resource/time window. (reservation_id, user_id, resource_id, start_time, end_time, status, created_at)
6. Loan / Checkout — a lending instance (for books, kits). (loan_id, resource_id, user_id, checkout_at, due_at, returned_at, status)
7. QRCode / Barcode — physical token linking to resource. (token_id, resource_id, payload)
8. CheckEvent — check-in/out record. (event_id, loan_or_reservation_id, user_id, type, timestamp)
9. HoldRequest — queued request for a resource copy. (hold_id, user_id, resource_id, placed_at, notified_at)
10. MaintenanceEvent — planned maintenance/cleaning for resources or rooms. (maintenance_id, resource_id, start_time, end_time, reason)
11. Notification — message (email/push/SMS). (notification_id, user_id, type, body, sent_at)
12. Policy — rules that govern loans and reservations (max duration, fines). (policy_id, resource_type, max_duration, fines_per_day)

Relationships:

- a. Department OWNS Resource — 1 Department → many Resources.
- b. Resource LOCATED_IN Location/Floorplan — 1 Resource → 1 Location (floor/building coordinates).
- c. User CREATES Reservation — 1 User → many Reservations.
- d. Reservation RESERVES Resource — 1 Reservation → 1 Resource (reservation refers to one resource instance or record).
- e. Reservation MAY_LEAD_TO Loan — 0..1 Loan created from a Reservation (e.g., reserved book is checked out).
- f. Resource HAS Loan history — 1 Resource → many Loan records over time.
- g. Resource HAS Token (QR/Barcode) — 1 Resource → 1 Token (or multiple tokens for multiple copies).
- h. User PLACES HoldRequest on Resource — 1 User → many HoldRequests; 1 Resource → many HoldRequests (queue).
- i. MaintenanceEvent BLOCKS Resource — 1 MaintenanceEvent → 1 Resource (blocks availability during the interval).
- j. Policy APPLIES_TO Resource type / Department — Policy scopes to Resource types or Departments (many-to-many where needed).
- k. User ASSIGNED Role / Permissions — RBAC mapping: User $\leftrightarrow$ Role (many-to-many via assignments).

1.4. Project schedule (Gantt chart)

- tasks with dependencies,
- start time and duration,
- resource allocation: staff, costs, equipment.

Overview: 6-week MVP. Tasks include dependencies, start week (W1 = project start), duration in weeks, primary assigned role, estimated staff-hours and estimated labour cost.

ID	Task	Duration	Depends on	Notes
T1	Requirements & concept finalization	1 wk	—	Define resource types and workflows
T2	Data model & backend setup (API, DB)	1 wk	T1	Entities, relations, policies
T3	Frontend layout & navigation	1 wk	T1	React base, pages
T4	Reservation & loan logic	1 wk	T2	Booking + checkout modules
T5	QR/Barcode scan & map view	1 wk	T4	Visual status + room locations
T6	Notifications & admin panel	0.5 wk	T4	Emails, overdue alerts
T7	Testing, polishing & deployment	1.5 wks	T5-T6	QA + Docker deploy

Team effort (approx.): 450 hours

Estimated cost (approx.): €8,000 – €10,000

Resources: shared laptops, test phones/tablets for QR scanning, cloud DB & hosting (optional).

1.5. Identification of existing or alternative solutions

Each competitor should be characterized with

- name of the project/application,
- main features,
- advantages,
- disadvantages.

LibCal (Springshare)

- **Main features:** Room & equipment booking, calendar integrations, event scheduling, waitlists, reporting, user notifications.
- **Advantages:** Designed for academic libraries/campuses; polished UX; good customer support and analytics.
- **Disadvantages:** Commercial (licensing costs); limited backend/customization; vendor lock-in for integrations.

Ex Libris Alma / Primo

- **Main features:** Full ILS/ERMS (cataloguing, acquisitions, circulation), discovery layer, integrated workflows for library operations.
- **Advantages:** Enterprise-grade library functionality; strong integration with consortia and library standards.
- **Disadvantages:** Very heavy/complex; high cost; overkill for simple multi-resource booking (not optimized for rooms/equipment).

Booked Scheduler (phpScheduleIt)

- **Main features:** Resource & room scheduling, recurring bookings, admin approvals, user accounts, calendar export.
- **Advantages:** Self-hostable, lightweight, mature open-source roots, fairly simple to customize.
- **Disadvantages:** UI looks dated; limited built-in support for mobile scanning (QR/barcode) and advanced reporting.

Skedda

- **Main features:** Space & desk booking, floorplans, recurring rules, pricing/rule engines, usage analytics.
- **Advantages:** Modern UI; strong for spaces and floorplans; easy to use for end users.
- **Disadvantages:** Proprietary; key features behind paid tiers; limited custom integrations for campus IT.

Koha / Evergreen (open-source ILSs)

- **Main features:** Cataloguing, circulation (loans/holds), patron management, reporting for library collections.
- **Advantages:** Free/open source; robust book-lending features and community support.
- **Disadvantages:** Library-centric (not built for rooms/equipment/real-time map + scanning); requires customization to support mixed resources.

MRBS (Meeting Room Booking System)

- **Main features:** Simple meeting/room booking, calendar view, basic user management.
- **Advantages:** Extremely lightweight; easy to self-host; minimal operational cost.
- **Disadvantages:** Very limited feature set (no loans, scanning, advanced policies or reporting).

Resource Guru

- **Main features:** Resource scheduling for teams, visual calendar, availability management, utilization reports.
- **Advantages:** Clean visual scheduling; good for staff/equipment allocation; simple billing/usage reporting.
- **Disadvantages:** Commercial SaaS; focused on team scheduling rather than public campus lending; limited physical check-in features.

1.6. Project context

- application context
- technological context
- organisational context
- legal context

Indicate aspects that are important for the product. It should lead to the identification of constraints of the project and the chances for its development.

Application context

- Campus-wide multi-resource service (books, rooms, equipment, vehicles, kits) for students, faculty and staff.
- Primary user needs: fast discovery, reliable booking/loan lifecycles, on-site confirmation (QR/barcode/ISBN), and timely notifications.
- Target delivery: an MVP that replaces fragmented point solutions and reduces double-bookings.

Technological context

- Required integrations: university SSO (SAML / OpenID Connect / LDAP), mail/notification gateways, and optional catalogue APIs.
- Platform constraints: web-first PWA with camera access for scanning; single-VM Docker deployment for the MVP.
- Non-functional targets: average search response < **500 ms** under test load; map/status propagation within **30 s** for 90% of events; HTTPS everywhere; basic observability.

Organisational context

- Key stakeholders: Library staff, Facility/Lab managers, University IT, Students' Union / Admin.
- Operational requirements: role-based administration, delegated approvals and owner workflows, clear handover documentation and runbooks.
- Deployment/ops: must comply with IT change-control and support procedures; include a simple rollback plan and documented deployment steps.

Legal & compliance context

- GDPR: perform a **DPIA**; apply data minimisation; record lawful bases for processing; obtain consent for notifications where required.
- Retention policy (examples to adopt): transaction logs **1 year**; backups retention configurable (minimum 30 days); user account data **2–5 years** depending on institutional policy.
- Accessibility: conform to **WCAG 2.1 AA** for student-facing UI.
- Auditability: keep tamper-evident transaction logs for reservations/loans and provide exportable reports for billing and discipline.

Security & resilience

- Authentication & identity: integrate with SSO (SAML/OIDC); support RBAC and administrative local override for emergency access.
- Transport & storage: **TLS** for all in-transit data; encryption at rest for sensitive fields (PII, auth tokens).
- Hardening: basic vulnerability scan before launch, dependency upgrades policy, and least-privilege service accounts.
- Availability: healthchecks and an emergency maintenance mode (graceful degradation if IdP or other critical integration is unavailable).
- Offline/poor-network handling: local queueing of scan/check events on clients with reliable retry/sync; conflict resolution strategy on sync.

Operations, monitoring & backups

- Monitoring: application error tracking (e.g., Sentry) + metrics and alerting (e.g., Prometheus/Grafana) for key KPIs (response times, queue length, scan success rate, job failures).
- Alerts & on-call: define escalation path (email/Slack) for high-severity alerts.
- Backups: daily incremental DB backups, weekly full snapshot; retention policy and restore test at least once per release cycle.
- Logging: centralised logs with retention aligned to audit policy; ensure logs are searchable for incident investigation.
- Runbooks: include deploy, rollback, backup restore, and IdP outage procedures.

Constraints (must manage)

- Time: pilot window **6–9 weeks**.
- Budget: small (target < **€10k**), prioritise open-source and containerised components.
- Devices: support modern campus phones and browsers (no mandatory native app).
- Security: SSO + TLS + least privilege + pre-launch vulnerability check.

Opportunities (advantages to exploit)

- Consolidate siloed tools into a single discovery + checkout experience for high user value.
- Rapid, low-cost MVP using containerised open-source building blocks.
- Differentiator: physical confirmation (QR/ISBN) + campus map visibility delivering immediate operational improvements.

Measurable success criteria (pilot)

- Functional: > **80%** of pilot users can complete a reservation + checkout without staff help.
- Reliability: end-to-end booking/checkout success rate ≥ **98%** in QA tests.

- Conflicts: booking conflicts during pilot < **5%** of attempts.
- Scanning: ≥ **95%** successful scan confirmation rate on supported devices.

1.7. Technologies used in the project

Justify the choice of technology. Focus on each element and provide the required information

- technology name,
- description of the technology,
- justification,
- key responsibilities in the project, e.g., front-end development,
- link to the technology description.

React (Frontend UI)

- **Description:** Component-based JavaScript library for building responsive single-page applications and PWAs.
- **Justification:** Fast developer productivity, mature ecosystem for PWAs and camera / device APIs (important for in-browser QR scanning), excellent reusability for component-driven UI (speeds delivery in a 6–9 week MVP).
- **Key responsibilities:** Main UI framework (pages, components, PWA shell, camera/scan integration, client-side state management).
- **Reference:** (react.dev)

TypeScript (Language for frontend & backend)

- **Description:** Statically-typed superset of JavaScript that transpiles to JS.
- **Justification:** Reduces runtime bugs, improves long-term maintainability in a team setting, enables safer refactor and clearer API contracts for an MVP intended to scale. Using TypeScript across frontend and Node.js backend shortens onboarding and reduces integration errors.
- **Key responsibilities:** Type-safe codebase for React components, API types, domain models and server code.
- **Reference:** (typescriptlang.org)

Tailwind CSS (Styling)

- **Description:** Utility-first CSS framework enabling rapid, consistent UI composition.
- **Justification:** Minimal design overhead, fast consistent responsive styling that saves implementation time and CSS maintenance costs for an MVP.
- **Key responsibilities:** Responsive layouts, component utility classes, accessible UI baseline.
- **Reference:** (tailwindcss.com)

Node.js + Express (Backend API & Services)

- **Description:** JavaScript runtime (Node.js) with minimal web framework (Express) for building REST/GraphQL APIs.
- **Justification:** Same language across stack (TypeScript) speeds development; Node is well-suited for I/O bound operations (notifications, scan confirmations) and pairs well with real-time patterns. Lightweight, easy to containerise for single-VM deploy.
- **Key responsibilities:** Business logic, REST/GraphQL endpoints, auth integration, background workers for notifications/overdue jobs.

- **Reference:** (nodejs.org)

PostgreSQL (Primary relational database)

- **Description:** Feature-rich, ACID-compliant open-source relational database.
- **Justification:** Strong transactional guarantees for loans/reservations, rich query capabilities for reporting/audit, extensibility for future needs (PostGIS if advanced mapping required). Reliable for production workloads on modest hardware.
- **Key responsibilities:** Persistent storage of users, resources, reservations, loans, policies, and audit logs.
- **Reference:** ([PostgreSQL](https://www.postgresql.org))

GitHub Actions (CI/CD)

- **Description:** Integrated CI/CD platform to build, test and deploy from the repository.
- **Justification:** Automates build/test/image publish and deployment to the pilot VM; inexpensive to operate and keeps the pipeline auditable for handover.
- **Key responsibilities:** Linting, unit/integration tests, Docker image build, deploy promotion to staging/production.
- **Reference:** ([GitHub Docs](https://docs.github.com))